

Functional data analysis

3. Functional data preparation

Alfredas Račkauskas, Jurgita Markevičiūtė

Lecture notes, 2026

Objectives:

- *to demonstrate how one moves from observed vectors to functional objects;*
- *to discuss the basics of smoothing (functional basis, splines, kernel and others);*
- *to demonstrate how basis expansions can be used to create and store functional objects.*

Outline of the lecture

Introduction

Smoothing

Basis functions method

Estimation

Least squares method

Estimation with Roughness Penalties

Kernel smoothing

Local polynomial smoothing

Introduction

- A fundamental idea of the FDA is that the objects we wish to study are (usually smooth) curves

$$(x_j(t), t \in \mathcal{T}), \quad j = 1, 2, \dots, n,$$

for which the values $x_j(t)$ exist at any point t , but are observed only at selected points $t = t_{ji}$, $i = 1, \dots, m_j$, which can be different for different curves.

- The time points $(t_{ji}, i = 1, \dots, m_j)$ are called the **design time points** (for x_j) and may be either equally spaced in the interval of interest, or be regarded as a random sample from a continuous design density, say, $g_j(t)$.

Each curve $x_j = (x_j(t), t \in \mathcal{T})$ is interpreted as a realization of the path of a random process $X_j = (X_j(t), t \in \mathcal{T})$.

- Observation $x_j(t_{ji})$ may be obtained either with some measurement error or without.
- Hence, we actually have data set

$$(t_{ji}, y_{ji}), \quad i = 1, \dots, m_j; \quad j = 1, \dots, n,$$

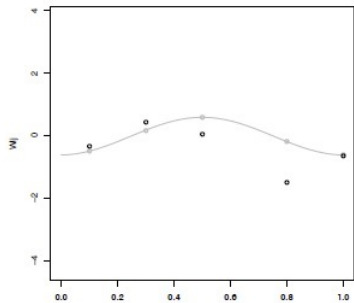
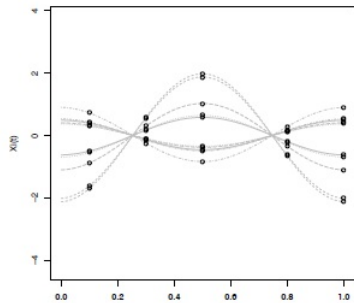
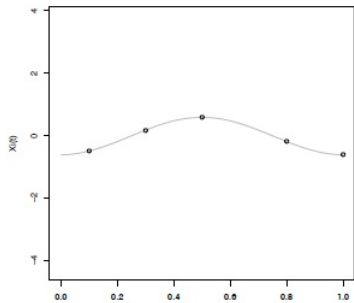
where either

$$y_{ji} = x_j(t_{ji}), \quad i = 1, \dots, m_j; \quad j = 1, \dots, n,$$

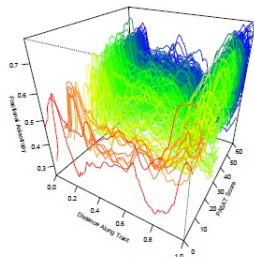
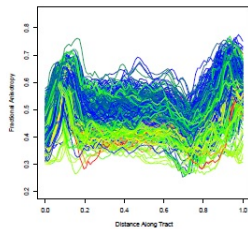
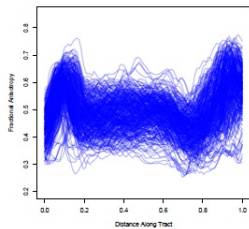
or

$$y_{ji} = x_j(t_{ji}) + \varepsilon_{ji}, \quad i = 1, \dots, m_j; \quad j = 1, \dots, n,$$

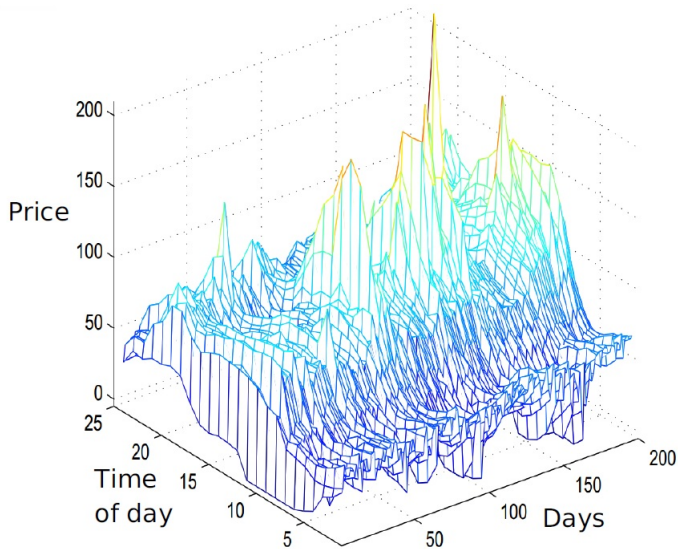
where for each j , $\varepsilon_{ji}, i = 1, \dots, m_j$, are the measurement errors that can be explained by the function x_j .



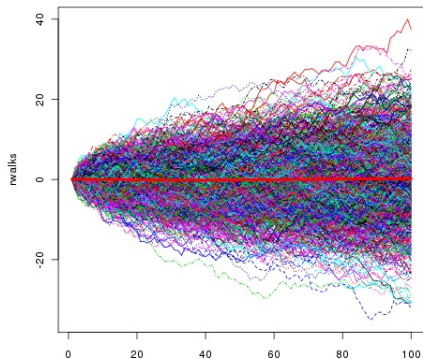
Raw data I



Raw data II



Raw data III



Simulated Wiener process

Smoothing

Reconstruction of the function $x_j(t)$ uses smoothing techniques.
This allow us to achieve the following goals:

- to reconstruct the individual functions in a real functional data set so that any reconstructed individual function can be evaluated at any desired resolution;
- to remove measurement errors as much as possible so that the variations of the reconstructed functional data mainly come from the between-subject variations,
- to improve statistical inference,
- others.

Basis functions method I

Consider reconstruction of one function $(x(t), t \in \mathcal{T})$ observed at the design time points $t_1, \dots, t_m \in \mathcal{T}$.

Observations $(t_j, y_j), j = 1, 2, \dots, m :$

$$y_j = x(t_j) + \varepsilon_j, \quad j = 1, 2, \dots, m.$$

- A *functional basis* is a set of known functions $\{\psi_k, k \in N\}$ such that any function from a certain class could be approximated by a linear combination of a sufficiently large number of basis functions.

Assumption

$$\begin{aligned} x(t) &\approx \sum_{k=1}^K \lambda_k \psi_k(t) \\ &= \boldsymbol{\lambda}' \boldsymbol{\psi}(t), \quad t \in \mathcal{T}. \end{aligned}$$

Basis functions method II

- The parameters $\lambda_1, \lambda_2, \dots, \lambda_K$ are the unknown coefficients of the expansion.
- λ stands for the vector of K coefficients and ψ denotes a vector of length K containing the basis functions:

$$\lambda = \begin{bmatrix} \lambda_1 \\ \vdots \\ \lambda_K \end{bmatrix}, \quad \psi(t) = \begin{bmatrix} \psi_1(t) \\ \vdots \\ \psi_K(t) \end{bmatrix}.$$

There are several different types of basis (Fourier, splines, wavelets, polynomial,...).



- How to choose K ?
- How to choose a functional basis from several different types?
- How to find/estimate $\lambda_1, \dots, \lambda_K$?

Functional basis examples

- Monomial: $1, t, t^2, t^3, \dots$;

$$x(t) \approx \lambda_0 + \lambda_1 t + \dots + \lambda_K t^K$$

- Exponential: $e^{b_1 t}, e^{b_2 t}, e^{b_3 t}, \dots$;

$$x(t) \approx \lambda_1 e^{b_1 t} + \dots + \lambda_K e^{b_K t}$$

- Polygonal: defining a function made up of straight line segments;
- Power: $t^{a_1}, t^{a_2}, \dots$, consisting of a sequence of possibly non-integer powers, including negative powers, of an argument t that is required to be positive;

$$x(t) \approx \lambda_1 t^{a_1} + \dots + \lambda_K t^{a_K}$$

- Fourier;
- Spline;
- others.

Basis objects: example

```
create.exponential.basis(rangeval=c(0,1),  
nbasis=NULL, ratevec=NULL, dropind=NULL,  
quadvals=NULL, values=NULL,  
basisvalues=NULL, names='exp', axes=NULL)
```

- **nbasis**

Default = if(is.null(ratevec)) 2 else
length(ratevec).

- **ratevec**

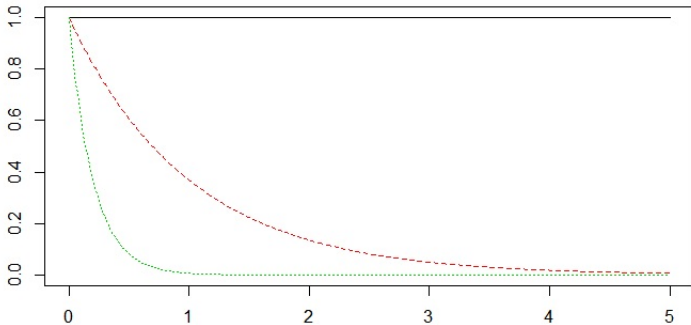
a vector of length nbasis of rate constants defining basis
functions of the form $\exp(\text{rate} \cdot x)$. Default = 0:(nbasis-1).

- **dropind**

vector of integers specifying the basis functions to be dropped,
if any.

Create and plot an exponential basis over interval $[0, 5]$ with basis functions 1 , $\exp(-t)$ and $\exp(-5t)$.

```
basisobj1 <- create.exponential.basis(c(0,5),3,c(0,-1,-5))  
plot(basisobj1)
```



Fourier basis

- Perhaps the best known basis expansion is provided by the Fourier series:

$$x(t) \approx \lambda_0 + \lambda_1 \sin(wt) + \lambda_2 \cos(wt) + \lambda_3 \sin(2wt) + \lambda_4 \cos(2wt) \\ + \cdots + \lambda_{2M-1} \sin(Mwt) + \lambda_{2M} \cos(Mwt)$$

composed by $K = 2M + 1$ basis functions:

$$\psi_0(t) = 1, \psi_{2r-1}(t) = \sin(rwt), \psi_{2r}(t) = \cos(rwt), \quad r = 1, 2, \dots, M.$$

- This basis is periodic, and the parameter w determines the period $2\pi/w$ which should be equal to the length of the interval $\mathcal{T}$ (hence, $w = 2\pi/T$ if $\mathcal{T} = [0, T]$).
- Only two pieces of information are required to define a Fourier basis system:
 - the number of basis functions K and
 - the period T .
- The value T can often default to the range of t values spanned by the data.

Fourier basis object

```
create.fourier.basis(rangeval=c(0, 1), nbasis=3,  
period=diff(rangeval))
```

- `nbasis`

- positive odd integer.

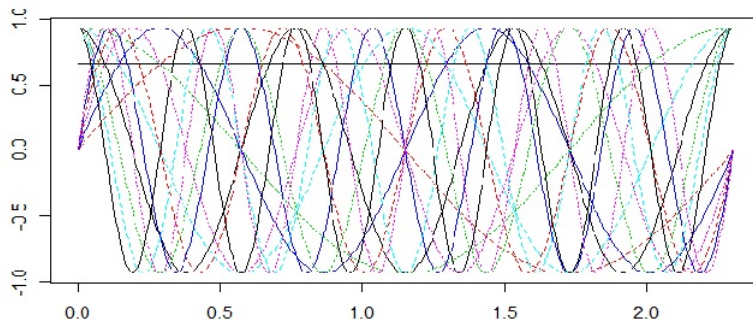
If an even number is specified, it is rounded up to the nearest odd integer to preserve the pairing of sine and cosine functions. An even number of basis functions only makes sense when there are always only an even number of observations at equally spaced points; that case can be accommodated using `dropind = nbasis-1` (because the bases are `const`, `sin`, `cos`, ...).

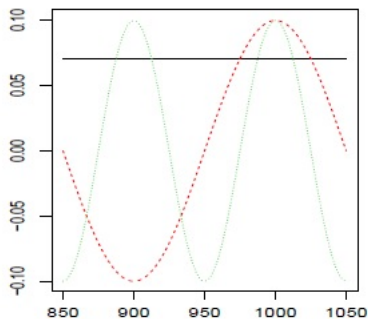
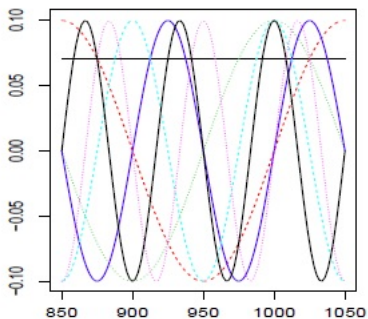
- `period`

- the width of any interval over which the Fourier functions repeat themselves or are periodic.

Examples

```
rangeval = c(0,2.3)
nbasis = 13
period = 2.3
fbasis = create.fourier.basis(rangeval,nbasis,period)
plot(fbasis)
```





Fourier basis with the first 7 basis function (left panel) and with the 1st, 3rd and 5th basis function (right panel).

- Derivative estimation of $x(t)$ is simple since

$$\frac{d}{dt} \sin(rwt) = rw \cos(rwt), \quad \frac{d}{dt} \cos(rwt) = -rw \sin(rwt).$$

- This implies that the Fourier expansion of $x'(t)$ has coefficients

$$(0, -w\lambda_2, w\lambda_1, -2w\lambda_4, 2w\lambda_3, \dots)$$

- and $x''(t)$ has coefficients

$$(0, -w^2\lambda_1, -w^2\lambda_2, -4w^2\lambda_3, -4w^2\lambda_4, \dots).$$

Spline basis

- Typically used for non-periodic data
- Spline functions are constructed by joining polynomials end-to-end at argument values called break points or knots.
- First, the interval is subdivided into a set of adjoining intervals separated the knots. Then a polynomial of order m (degree $m - 1$) is defined for each interval.
- To make the resulting piecewise polynomial smooth, two adjoining polynomials are constrained to have their values and all their derivatives up to order $m - 2$ match at the point where they join.

- In spline smoothing, one specifies a group of locations, say,

$$\tau_0, \tau_1, \tau_2, \dots, \tau_m,$$

in the range of interest, say, an interval $[a, b]$, where

$$a = \tau_0 \leq \tau_1 \leq \dots \leq \tau_m = b.$$

- These locations are known as *knots* and $\{\tau_0, \tau_1, \dots, \tau_m\}$ is the knot vector.
- These knots divide the interval $[a, b]$ into m subintervals:

$$[\tau_r, \tau_{r+1}], \quad r = 0, 1, \dots, m-1.$$

- If a knot τ_i appears k times (i.e., $\tau_i = \tau_{i+1} = \dots = \tau_{i+k-1}$), where $k > 1$, τ_i is a multiple knot of multiplicity k , written as $\tau_i(k)$. Otherwise, if τ_i appears only once, it is a simple knot.

- If the knots are equally spaced (i.e., $\tau_{i+1} - \tau_i$ is a constant for $0 \leq i \leq m - 1$), the knot vector or the knot sequence is said to be uniform; otherwise, it is non-uniform.
- All spline basis functions are supposed to have their domain on $[\tau_0, \tau_m]$. We shall use $\tau_0 = 0$ and $\tau_m = 1$ frequently so that the domain is the closed interval $[0, 1]$.

- A spline smoothing can be constructed using the following so-called *k'th degree truncated power basis* with knots $\tau_0, \tau_1, \tau_2, \dots, \tau_m$:

$$1, t, \dots, t^k, (t - \tau_1)_+^k, \dots, (t - \tau_{m-1})_+^k,$$

where $w_+^k = [w_+]^k$ denotes the power k of the positive part of w with $w_+ = \max\{0, w\}$.

- Using the truncated power basis, a spline approximation can be expressed as

$$x(t) \approx \sum_{j=0}^k \lambda_j t^j + \sum_{r=1}^{m-1} \lambda_{k+r} (t - \tau_r)_+^k,$$

where $\lambda_0, \lambda_1, \dots, \lambda_{k+m-1}$ are the associated coefficients.

- Within any subinterval or local neighbourhood $[\tau_r, \tau_{r+1})$, we have

$$x(t) \approx z(t) := \sum_{j=0}^k \lambda_j t^j + \sum_{\ell=1}^r \lambda_{k+\ell} (t - \tau_\ell)^k,$$

which is a k 'th degree polynomial.

- However, for $r = 1, 2, \dots, k$, we have

$$z^{(k)}(\tau_r-) = k! \left(\lambda_k + \sum_{\ell=1}^{r-1} \lambda_{k+\ell} \right),$$

and

$$z^{(k)}(\tau_r+) = k! \left(\lambda_k + \sum_{\ell=1}^r \lambda_{k+\ell} \right).$$

- It follows that

$$z^{(k)}(\tau_r+) - z^{(k)}(\tau_r-) = k! \lambda_{k+r}.$$

- That is, $z^{(k)}(t)$ jumps at the knot τ_r with amount $k! \lambda_{k+r}$ for $r = 1, 2, \dots, m-1$.

B-spline basis I

- While truncated power basis functions are natural, a much better computational choice, both for speed and numerical accuracy, is the *B-spline* basis.
- To define *B-spline* basis functions, we need one more parameter, the **degree of these basis** functions, p .
- The i 'th *B-spline* basis function of degree p , written as $B_{i,p}(t)$, is defined recursively as follows:

$$B_{i,0}(t) = \begin{cases} 1 & \text{if } \tau_i \leq t < \tau_{i+1} \\ 0 & \text{otherwise.} \end{cases}$$

and

$$B_{i,p}(t) = \frac{t - \tau_i}{\tau_{i+p} - \tau_i} B_{i,p-1}(t) + \frac{\tau_{i+p+1} - t}{\tau_{i+p+1} - \tau_{i+1}} B_{i+1,p-1}(t).$$

B-spline basis II

- Number of basis functions $K = m + p$.
- Expansion

$$x(t) \approx \sum_{j=0}^{m+p-1} \lambda_j B_{j,p}.$$

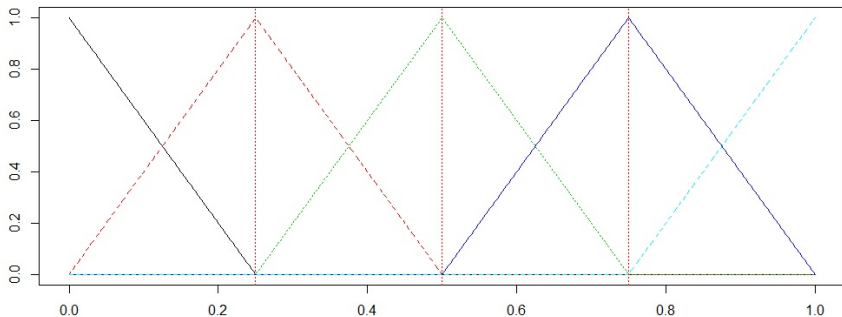
- If the degree is zero (i.e., $p = 0$), the basis functions $B_{i,p}(t) = B_{i,0}(t)$ are all step functions. That is, basis function $B_{i,0}(t)$ is 1 if t is in the i -th knot span $[\tau_i, \tau_{i+1})$.

B-spline basis object

```
create.bspline.basis(rangeval=NULL,  
nbasis=NULL, norder=4, breaks=NULL,  
dropind=NULL, quadvals=NULL, values=NULL,  
basisvalues=NULL, names="bspl")}}
```

Example

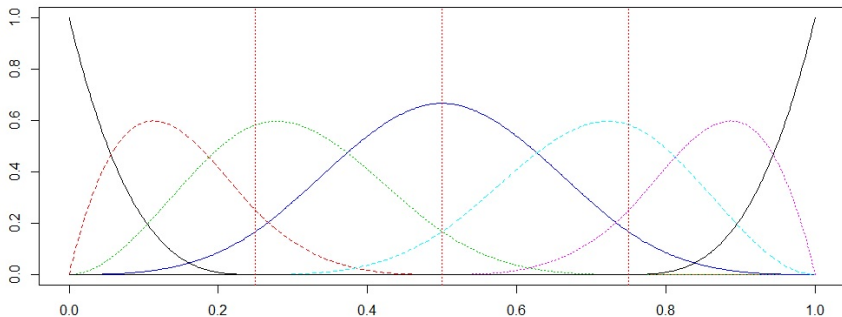
```
rangval = c(0,1)
breaks = c(0, 0.25, 0.5, 0.75, 1)
norder = 2
bbasis = create.bspline.basis(rangval,norder=norder,
breaks=breaks)
plot(bbasis)
```



$$K = nbreaks + norder - 2 = 5 + 2 - 2 = 5$$

Example

```
rangval = c(0,1)
breaks = c(0, 0.25, 0.5, 0.75, 1)
norder = 4
bbasis = create.bspline.basis(rangval,norder=norder,
breaks=breaks)
plot(bbasis)
```



$$K = nbreaks + norder - 2 = 5 + 4 - 2 = 7$$

Estimation: least squares method I

- Recall we observe (t_j, y_j) , $j = 1, 2, \dots, m$ and consider the model

$$y_j = x(t_j) + \varepsilon_j, \quad j = 1, \dots, m,$$

- where

$$x(t) \approx \sum_{k=1}^K \lambda_k \psi_k(t).$$

- We estimate $\lambda_1, \dots, \lambda_K$ from the regression

$$y_j = \sum_{k=1}^K \lambda_k \psi_k(t_j) + \varepsilon_j, \quad j = 1, \dots, m.$$

obtained as approximation to

$$y_j = x(t_j) + \varepsilon_j, \quad j = 1, \dots, n,$$

Estimation: least squares method II

by minimizing the least squares criterion

$$LSE(\boldsymbol{\lambda}) = \sum_{j=1}^m \left[y_j - \sum_{k=1}^K \lambda_k \psi_k(t_j) \right]^2$$

- In matrix form

$$LSE(\boldsymbol{\lambda}) = (\mathbf{y} - \boldsymbol{\Psi}\boldsymbol{\lambda})'(\mathbf{y} - \boldsymbol{\Psi}\boldsymbol{\lambda}),$$

where $\boldsymbol{\Psi}$ is the $m \times K$ matrix $\boldsymbol{\Psi} = [\psi_k(t_i)]$,
 $\mathbf{y} = (y_1, \dots, y_m)'$. $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_K)'$.

- Hence, the OLS estimator of $\boldsymbol{\lambda}$ is

$$\hat{\boldsymbol{\lambda}} = (\boldsymbol{\Psi}'\boldsymbol{\Psi})^{-1}\boldsymbol{\Psi}'\mathbf{y}.$$

Functional Data Objects

- A functional data object is just a basis object and a vector of coefficients.

$$(\lambda_1, \dots, \lambda_K), \text{ basisObj}$$

- Once we define the functional data object, we can treat them like data.
- 'plot' is defined for functional data objects. By default it uses the values of function at 201 equally-spaced points.
- If you have a very complex function, this may not be enough.

`fd(coef=NULL, basisobj=NULL, fdnames=NULL)`

- `coef`

a vector, matrix, or three-dimensional array of coefficients. The first dimension (or elements of a vector) corresponds to basis functions. A second dimension corresponds to the number of functional observations, curves or replicates.

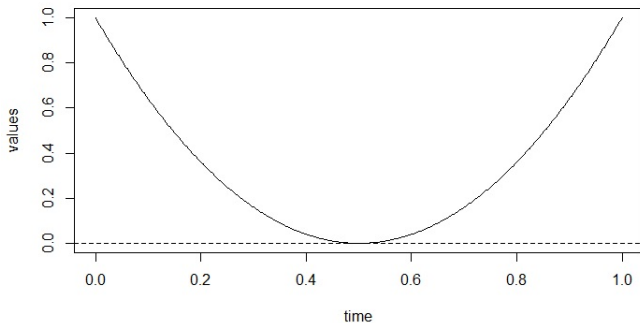
- `basisobj`

a functional basis object defining the basis

- `fdnames`

A list of length 3, each member being a string vector containing labels for the levels of the corresponding dimension of the discrete data. The first dimension is for argument values, and is given the default name "time", the second is for replications, and is given the default name "reps", and the third is for functions, and is given the default name "values".

```
rangeval=c(0,1)
mon3 = create.monomial.basis(nbasis=3)
parab4.5 = fd(c(1, -4, 4), mon3)
plot(parab4.5)
```



$$\text{parab}(t) = 1 - 4t + 4t^2, \quad t \in [0, 1].$$

fd objects by smoothing

`smooth.basis(y, fdParobj)`

Discrete observations of one or more curves are fit with a set of smooth curves, each defined by an expansion in terms of user-selected basis functions.

- `y`
a set of values of curves at discrete sampling points or argument values.
- `fdParobj`
a functional parameter object, a functional data object or a functional basis object. In the simplest case, `fdParobj` may be a functional basis object with class "basisfd" defined previously by one of the "create" functions.

Example

- Set up 51 observation points equally spaced between 0 and 1
`n = 51`
`argvals = seq(0,1,len=n)`
- The true curve values are sine function values with period 1/2
`x = sin(4*pi*argvals)`
- Add independent Gaussian errors with std. dev. 0.2 to the true values
`sigerr = 0.2`
`y = x + rnorm(x)*sigerr`

- Set up a B -spline basis system of order 4 (piecewise cubic) and with knots at 0, 0.1, ..., 0.9 and 1.0, and plot the basis functions

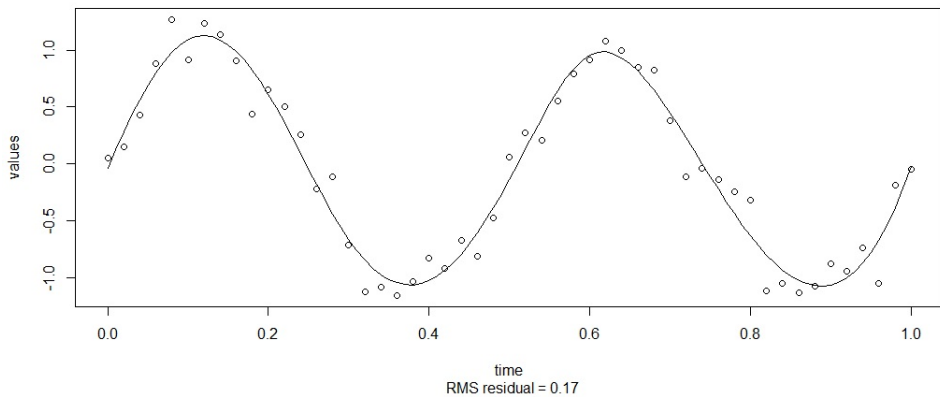
```
nbasis = 13
```

```
basisobj = create.bspline.basis(c(0,1),nbasis)  
plot(basisobj)
```

- Smooth the data, outputting only the functional data object for the fitted curve. Note that in this simple case we can supply the basis object as the "fdParobj" parameter

```
ys = smooth.basis(argvals=argvals, y=y,  
fdParobj=basisobj)  
xfd = ys$fd
```

- Plot the curve along with the data
`plotfit.fd(y, argvals, xfd)`



The D operator

We use notation that for a function $x(t)$

$$Dx(t) = \frac{d}{dt}x(t).$$

We can also define further derivatives in terms of power D :

$$D^2x(t) = \frac{d^2}{dt^2}x(t), \dots, D^kx(t) = \frac{d^k}{dt^k}x(t), \dots$$

- $Dx(t)$ is the instantaneous slope of $x(t)$; $D^2x(t)$ is its curvature.

Estimation with Roughness Penalties I

- The roughness penalty approach uses a large number of basis functions, possibly extending to one basis function per observation and even beyond, but at the same time imposing smoothness by penalizing some measure of function complexity
- One defines a measure of the roughness of the fitted curve, and then minimizes a fitting criterion that trades off curve roughness against lack of data fit.
- The square of the second derivative $[D^2x(t)]^2$ of a function x at argument value t is often called its curvature at t , since a straight line, which we all tend to agree has no curvature, has second derivative zero.
- Consequently, a measure of a function's roughness is the integrated squared second derivative or total curvature

$$PEN_2(x) = \int [D^2x(t)]^2 dt.$$

Estimation with Roughness Penalties II

- The classical measure of roughness is

$$PEN_2(x) = \int [D^2x(s)]^2 ds.$$

$[D^2x(s)]^2$ measures the curvature in x at s . This penalty measures total curvature.

- Curvature in acceleration:

$$PEN_4(x) = \int [D^4x(s)]^2 ds.$$

- These two penalties also define what we mean by "smooth"; any function that has zero penalty is "hyper-smooth." A straight line for the classic, a cubic polynomial for the acceleration penalty.

Harmonic acceleration

- If the process is periodic, it is natural to think of a constant + sinusoid as "hyper-smooth".
- This suggests that we use

$$PEN_H(x) = \int [D^3x(s) + \omega^2 Dx(s)]^2 ds$$

where $2\pi/\omega$ is the period, since zero roughness implies that x is of the form

$$x(t) = c_1 + c_2 \sin \omega t + c_3 \cos \omega t,$$

where ω is the period.

- The functions 1 , $\sin(\omega t)$, and $\cos(\omega t)$ all have zero penalties, as does any linear combination of them.
- Writing

$$Lx(s) = D^3x(s) + \omega^2 Dx(s)$$

we have

$$PEN_H(x) = \int [Lx(s)]^2 ds$$

Penalized least squares estimation

Recall

$$LSE(\lambda) = \sum_{j=1}^m \left[y_j - \sum_{k=1}^K \lambda_k \psi_k(t_j) \right]^2$$

The penalized least squares criterion is

$$PENLSE_{\lambda}(x) = \sum_{j=1}^m \left[y_j - \sum_{k=1}^K \lambda_k \psi_k(t_j) \right]^2 + \lambda PEN\left(\sum_{k=1}^K \lambda_k \psi_k\right).$$

- Smoothing parameter λ controls the amount of roughness.
 - As $\lambda \rightarrow 0$, roughness matters less and less, and $x(t)$ fits the data better and better.
 - As $\lambda \rightarrow \infty$, roughness matters more and more, and $x(t)$ becomes more and more "hyper-smooth."

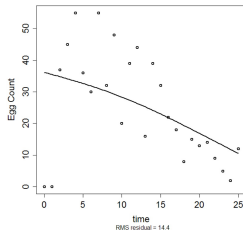
What do we mean by smoothness?

Some things are fairly clearly smooth:

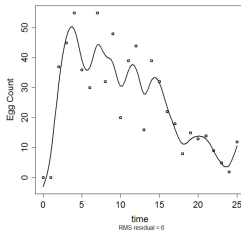
- constants
- straight lines

What we really want to do is eliminate small "wiggles" in the data while retaining the right shape.

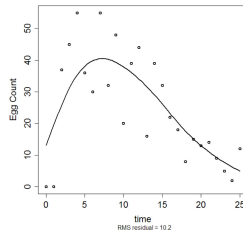
Too smooth



Too rough



Just right



- Whatever roughness penalty we use, we add some multiple of it to the error sum of squares to define the compound fitting criterion.
- For example, using $PEN_2(x)$ gives us the following:

$$F(\lambda) = \sum_j [y_j - \hat{x}(t_j)]^2 + \lambda \int [D^2 \hat{x}(t)]^2 dt.$$

Choosing smoothing parameter λ

Cross-validation for choosing the smoothing parameter λ

In cross-validation, we

- set aside a subset of data, the validation sample
- call the balance of the data the training sample
- fit the model to the training sample
- assess fit to the validation sample
- choose the λ value that gives the best fit

We can also, for a sequence of values of λ ,

- set aside each observation (t_j, y_j) in turn
- fit the data with the rest of the sample,
- sum fits to the left out values to get a cross-validated error sum of squares $CV(\lambda)$.
- select the λ value that minimizes $CV(\lambda)$.

Generalized cross-validation

- Cross-validation is time-consuming, and tends too often to under-smooth the data.
- The generalized cross-validation criterion is

$$GCV(\lambda) = \frac{m}{m - df(\lambda)} \cdot \frac{SSE}{m - df(\lambda)},$$

where df is the equivalent degrees of freedom of the smoothing operator.

- The right factor is just the unbiased estimate s_{ε}^2 of residual variance familiar in regression analysis.
- The left factor further "discounts" this measure further to allow for the influence of optimizing with respect to λ .

Kernel smoothing

- We assume a non-parametric regression model

$$y_i = x(t_i) + \varepsilon_i, \quad i = 1, \dots, m.$$

- The Nadaraya-Watson kernel estimator of $x(t)$ is defined by

$$\hat{x}_h(t) = \frac{\sum_{i=1}^m K((t_i - t)/h)y_i}{\sum_{i=1}^m K((t_i - t)/h)},$$

where K is a kernel function and a constant $h > 0$ called the *bandwidth* or *smoothing parameter*.

The kernel function $K(\cdot)$ is usually a probability density function.

- Gaussian:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\{-u^2/2\}.$$

- Epanechnikov:

$$K(u) = \frac{3}{4} \mathbf{1}_{[-1,1]}(u)(1 - u^2).$$

- Uniform:

$$K(u) = \frac{1}{2} \mathbf{1}_{[-1,1]}(u).$$

- Cosine:

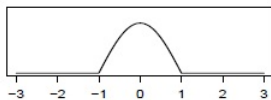
$$K(u) = \frac{\pi}{4} \mathbf{1}_{[-1,1]}(u) \cos(\pi u/2).$$

- Quadratic:

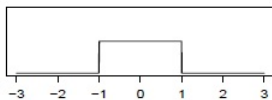
$$K(u) = \frac{15}{16} \mathbf{1}_{[-1,1]}(u) (1 - u^2)^2.$$

- Triweight:

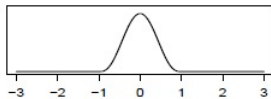
$$K(u) = \frac{35}{32} \mathbf{1}_{[-1,1]}(u) (1 - u^2)^3.$$



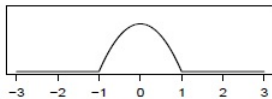
a: Cosine kernel



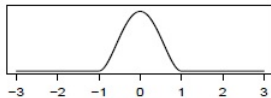
b: Uniform kernel



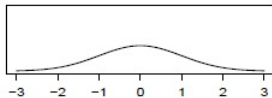
c: Triweight kernel



d: Epanechnikov kernel



e: Quartic kernel



f: Gaussian kernel

Figure 3.3: Symmetric Kernels

- When K is uniform kernel, the Nadaraya-Watson estimator is exactly the local average of y_i 's that are within the local neighborhood $I_h(t) := [t - h, t + h]$:

$$\hat{x}_h(t) = \frac{\sum_{i=1}^m I_{I_h(t)}(t_i) y_i}{\sum_{i=1}^m I_{I_h(t)}(t_i)} = \left\{ \sum_{t_i \in I_h(t)} y_i \right\} / m_h(t),$$

where $m_h(t)$ denotes the number of the observations falling into the local neighborhood $I_h(t)$.

- When t is a boundary point of $\mathcal{T}$, fewer design points are within the neighborhood $I_h(t)$ so that $\hat{x}_h(t)$ has a slower convergence rate than the case where t is an interior point of $\mathcal{T}$.

Local Polynomial Kernel Smoothing I

- The main idea of local polynomial kernel (LPK) smoothing is to locally approximate the underlying function $x(t)$ by a polynomial of some degree.
- Its foundation is Taylor expansion, which states that any smooth function can be locally approximated by a polynomial of some degree.
- Specifically, let t_0 be an arbitrary fixed time point where the function $x(t)$ will be estimated.
- **Assume** $x(t)$ has a $(p + 1)$ 'st continuous derivative for some integer $p \geq 0$ at t_0 .

Local Polynomial Kernel Smoothing II

- By Taylor expansion, $x(t)$ can be locally approximated by a polynomial of degree p , that is,

$$x(t) \approx x(t_0) + (t - t_0) \frac{x^{(1)}(t_0)}{1!} + (t - t_0)^2 \frac{x^{(2)}(t_0)}{2!} + \cdots \\ \cdots + (t - t_0)^p \frac{x^{(p)}(t_0)}{p!},$$

in a neighbourhood of t_0 that allows the above expansion where $x^{(r)}(t_0)$ denotes the r 'th derivative of $x(t)$ at t_0 .

- Set $\beta_r = x^{(r)}(t_0)/r!$, $r = 0, \dots, p$.
- Then we have

$$x(t) \approx \beta_0 + \beta_1(t - t_0) + \beta_2(t - t_0)^2 + \cdots + \beta_p(t - t_0)^p.$$

Local Polynomial Kernel Smoothing III

- Let $\hat{\beta}_r, r = 0, 1, 2, \dots, p$, be the minimizers of the following weighted least squares (WLS) criterion:

$$\sum_{i=1}^n \left\{ y_i - [\beta_0 + (t_i - t_0)\beta_1 + \dots + (t_i - t_0)^p \beta_p] \right\}^2 K_h(t_i - t_0),$$

where

$$K_h(\cdot) = K(\cdot/h)/h,$$

is obtained by rescaling a kernel function $K(\cdot)$ with a constant $h > 0$ called the *bandwidth* or *smoothing parameter*.

- This is a weighted least squares problem where the weights are given by the kernel functions $K((t_i - t_0)/h)$.
- The bandwidth h is mainly used to specify the size of the local neighborhood, namely,

$$I_h(t_0) = [t_0 - h, t_0 + h],$$

where the local fit is conducted.

Local Polynomial Kernel Smoothing IV

- The kernel function, $K(\cdot)$, determines how observations within $[t_0 - h, t_0 + h]$ contribute to the fit at t_0 .

It is convenient to define the following vectors and matrices:

$$\mathbf{X}_{t_0} = \begin{pmatrix} 1 & (t_1 - t_0) & \cdots & (t_1 - t_0)^p \\ \vdots & \vdots & \vdots & \vdots \\ 1 & (t_n - t_0) & \cdots & (t_n - t_0)^p \end{pmatrix}$$

$$\mathbf{y} = (y_1, y_2, \dots, y_n)', \quad \boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)'$$

and

$$\mathbf{W}_{t_0} = \text{diag}(K_h(t_1 - t_0), \dots, K_h(t_n - t_0)),$$

be the design matrix and the weight matrix for the LPK fit around t_0 .

- Denote the estimate of the r 'th derivative $x^{(r)}(t_0)$ as $\hat{x}_h^{(r)}(t_0)$.

Local Polynomial Kernel Smoothing V

- Then

$$\hat{x}_h^{(r)}(t_0) = r! \hat{\beta}_r, \quad r = 0, 1, \dots, p.$$

- In particular, the resulting p 'th degree LPK estimator of $x(t_0)$ is

$$\hat{x}_h(t_0) = \hat{\beta}_0.$$

- An explicit expression for $\hat{x}_h^{(r)}(t_0)$ is useful and can be made by matrix notation.

Local Polynomial Kernel Smoothing VI

- Let

$$\mathbf{X} = \begin{pmatrix} 1 & (t_1 - t_0) & \cdots & (t_1 - t_0)^p \\ \vdots & \vdots & \vdots & \vdots \\ 1 & (t_n - t_0) & \cdots & (t_n - t_0)^p \end{pmatrix}$$

and

$$\mathbf{W} = \text{diag}(K_h(t_1 - t_0), \dots, K_h(t_n - t_0)),$$

be the design matrix and the weight matrix for the LPK fit around t_0 .

- The least squares problem is then to minimize the weighted sum-of-squares function

$$(\mathbf{y} - \mathbf{X}_{t_0}\beta)' \mathbf{W}_{t_0} (\mathbf{y} - \mathbf{X}_{t_0}\beta),$$

with respect to the parameters β .

Local Polynomial Kernel Smoothing VII

- The solution is

$$\hat{\beta} = (\mathbf{X}'_{t_0} \mathbf{W}_{t_0} \mathbf{X}_{t_0})^{-1} \mathbf{X}'_{t_0} \mathbf{W}_{t_0} \mathbf{y}$$

provided $\mathbf{X}'_{t_0} \mathbf{W}_{t_0} \mathbf{X}_{t_0}$ is a nonsingular matrix.

- The quantity $x(t_0)$ is then estimated by the fitted intercept parameter (i.e. by $\hat{\beta}_0$) as this defines the position of the estimated local polynomial curve at the point t_0 .
- By varying the value of t_0 , we can build up an estimate of the function $x(t)$ over the range of $t \in \mathcal{T}$.
- We have

$$\hat{x}_h(t_0) = \mathbf{e}'_1 \hat{\beta} = \mathbf{e}'_1 (\mathbf{X}'_{t_0} \mathbf{W}_{t_0} \mathbf{X}_{t_0})^{-1} \mathbf{X}'_{t_0} \mathbf{W}_{t_0} \mathbf{y}$$

where the vector $\mathbf{e}_1$ is of length $p+1$ and has a 1 in the first position and 0's elsewhere.

$$\hat{x}_h^{(r)}(t_0) = r! \mathbf{e}_{r+1,p+1}^T \mathbf{S}_n^{-1} \mathbf{T}_n \mathbf{y},$$

Local Polynomial Kernel Smoothing VIII

where $\mathbf{e}_{r+1,p+1}$ denotes a $(p+1)$ -dimensional unit vector whose $(r+1)$ 'st entry is 1 and the other entries are 0, and

$$\mathbf{S}_n = \mathbf{X}^T \mathbf{W} \mathbf{X}, \mathbf{T}_n = \mathbf{X}^T \mathbf{W}.$$

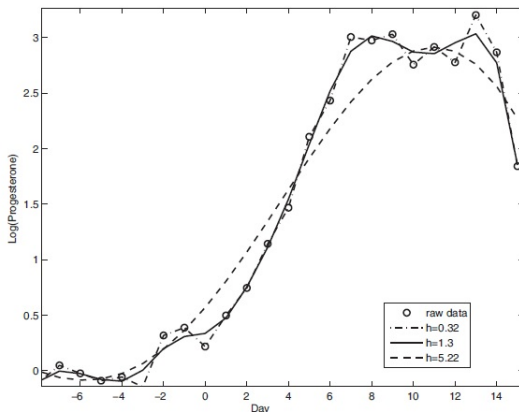
If $r = 0$ we have

$$\hat{x}_h(t_0) = \sum_{i=1}^n K_h^n(t_i - t_0) y_i,$$

where $K_h^n(t_i - t_0) = \mathbf{e}_{1,p+1}^T \mathbf{S}_n^{-1} \mathbf{T}_n \mathbf{e}_{i,n}$.

Selecting a Good Bandwidth

- In most applications, the kernel function $K(\cdot)$ can be simply chosen as the uniform kernel or the Gaussian kernel.
- It is more crucial to choose the bandwidth h .



- For a given p and a fixed kernel function $K(\cdot)$, a good bandwidth h should be chosen to trade off between the goodness of fit and the model complexity.
- Better goodness of fit often means less bias while smaller model complexity usually means less rough or smaller variance.
- To better choose the bandwidth h in kernel smoothing, we can use a generalized cross-validation (GCV) rule.
- In the literature, many other bandwidth selectors are available.

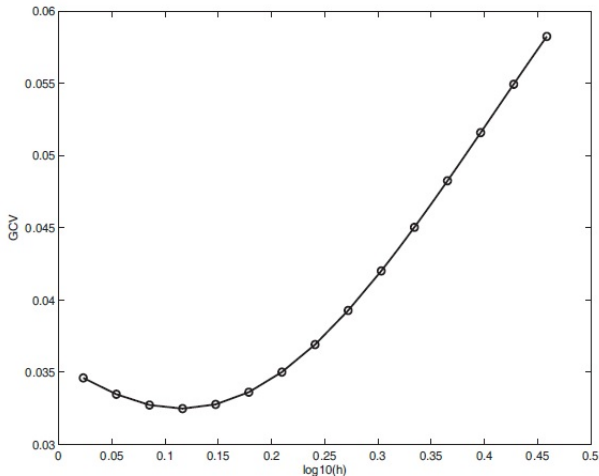


Figure displays the GCV curve versus bandwidth h in log 10- scale for selecting a good bandwidth for the local quadratic kernel fit with the Gaussian kernel. The associated optimal bandwidth is $h^* = 1.3$, associated with $\log(h^*) = 0.1139$.

- It is well known that the LPK smoother is not robust in the presence of outliers.
- To overcome the aforementioned problem, one can use the following robust LPK smoothing algorithm:

Step 0. Set $y_i^0 = y_i, i = 1, 2, \dots, n$.

Step 1. Compute the LPK fit to the data $(t_i, y_i^0), i = 1, 2, \dots, n$.
Record the fitted responses $\hat{y}_i^0, i = 1, 2, \dots, n$ and the associated residuals $\hat{\varepsilon}_i = y_i - \hat{y}_i^0, i = 1, 2, \dots, n$.

Step 2. Compute the standard deviation, $\hat{\sigma}$, of the residuals $\hat{\varepsilon}_i, i = 1, 2, \dots, n$.

Step 3. Set $y_i^0 = y_i$ when $|\hat{\varepsilon}_i| < 2\hat{\sigma}$ and set $y_i^0 = \hat{y}_i^0$ when $|\hat{\varepsilon}_i| \geq 2\hat{\sigma}$.

Step 4. Repeat Steps 1, 2, and 3 until convergence.